

Document Version	1.0 — Full Implementation Guide
Project Title	ClassSense: Real-Time Classroom Engagement & Emotion Monitoring System
Project Period	October 2025 – June 2026 (FYP-1 complete, FYP-2 in progress)
Purpose	Step-by-step guide to build ClassSense from scratch to a working MVP — readable by developer and any AI assistant
Technology Stack	Python 3.10, FastAPI, Next.js 14, PostgreSQL, OpenCV, MediaPipe, PyTorch, SORT, ReportLab
Methodology	Agile — incremental phases, each with a testable deliverable

Table of Contents

Phase 0	Project Setup & Prerequisites	<i>Environment, repo structure, datasets, tools</i>
Phase 1	ML Pipeline — Face Detection	<i>Detect faces in every video frame using MediaPipe</i>
Phase 2	ML Pipeline — Face Tracking	<i>Assign persistent anonymous IDs across frames</i>
Phase 3	ML Pipeline — Emotion Recognition	<i>Classify each face: attentive / confused / distracted</i>
Phase 4	ML Pipeline — Engagement Scoring	<i>Compute class-wide engagement percentage</i>
Phase 5	ML Pipeline — Gaze Estimation	<i>Head pose to determine attention direction</i>
Phase 6	Backend — FastAPI Server	<i>API endpoints + WebSocket for live streaming</i>
Phase 7	Database — PostgreSQL Schema	<i>Store sessions, analytics, summaries</i>
Phase 8	Frontend — Next.js Dashboard	<i>Live charts, session management, UI screens</i>
Phase 9	Integration — Connecting Everything	<i>Wire ML → FastAPI → WebSocket → Next.js</i>
Phase 10	Model Fine-tuning	<i>Upgrade from quick-start to your own trained model</i>
Phase 11	Reports — PDF & CSV Export	<i>Generate downloadable session reports</i>
Phase 12	Authentication — JWT Login	<i>Instructor / admin login with secure tokens</i>
Phase 13	Docker & Deployment	<i>Package everything to run on any machine</i>
Phase 14	Testing & Evaluation	<i>Accuracy, latency, usability tests for FYP submission</i>
Appendix A	Dataset Guide	<i>Where to get data, how to use it</i>
Appendix B	Folder Structure Reference	<i>Every file and folder explained</i>
Appendix C	AI Assistant Instructions	<i>How to use this doc with ChatGPT / Claude / Cursor</i>

Project Setup & Prerequisites

Understan

What Is ClassSense? (Read This First)

ClassSense is a web application that watches a classroom camera feed and tells the instructor — in real time — how engaged the students are. It does this by:

- Finding every student face in the video frame
- Tracking each face across time with an anonymous ID (no names, no identity stored)
- Analysing each face to detect the student's emotion (attentive, confused, or distracted)
- Estimating whether the student is looking forward or looking away
- Combining these signals into a single class engagement percentage (e.g. 87% engaged)
- Showing this data live on a web dashboard the instructor can view during the lecture
- Saving a summary report at the end of each session

GOLDEN RULE FOR THIS ENTIRE PROJECT: Build one piece, make it work, then move on. Never build two unfinished things at the same time. This document is a guide to produce something you can actually run and see working before moving on.

System Architecture Overview

The full system has three separate parts that talk to each other:

Part	Name	Language	What It Does
1 — ML Pipeline	Computer Vision Engine	Python	Reads video frames, detects faces, tracks them, recognises emotions, computes engagement score
2 — Backend	FastAPI Server	Python	Receives analytics from ML pipeline, stores to database, sends live data to dashboard via WebSocket
3 — Frontend	Next.js Dashboard	TypeScript/React	Shows live engagement charts, lets instructor start/end sessions, download reports

Folder Structure — Create This First

```
classsense/  
  ■■■ ml/ # All computer vision & AI code  
  ■ ■■■ detection/  
    ■ ■ ■■■ detector.py # Face detection using MediaPipe
```

```
■ ■ ■ ■ tracking/
■ ■ ■ ■ tracker.py # Face tracking using DeepSORT
■ ■ ■ ■ emotion/
■ ■ ■ ■ classifier.py # Emotion recognition model
■ ■ ■ ■ model_weights/ # Saved .pth model files
■ ■ ■ ■ gaze/
■ ■ ■ ■ estimator.py # Head pose / gaze estimation
■ ■ ■ ■ engagement/
■ ■ ■ ■ scorer.py # Combines emotion+gaze → score
■ ■ ■ ■ pipeline.py # Ties all ML steps together
■ ■ ■ ■ backend/
■ ■ ■ ■ main.py # FastAPI app entry point
■ ■ ■ ■ routers/
■ ■ ■ ■ sessions.py # Session start/end endpoints
■ ■ ■ ■ analytics.py # Analytics & report endpoints
■ ■ ■ ■ models/
■ ■ ■ ■ session.py # SQLAlchemy database models
■ ■ ■ ■ schemas.py # Pydantic request/response schemas
■ ■ ■ ■ services/
■ ■ ■ ■ ml_runner.py # Calls ml/pipeline.py
■ ■ ■ ■ report.py # PDF/CSV report generation
■ ■ ■ ■ database.py # DB connection setup
■ ■ ■ ■ auth.py # JWT authentication
■ ■ ■ ■ frontend/
■ ■ ■ ■ app/
■ ■ ■ ■ page.tsx # Login page
■ ■ ■ ■ dashboard/
■ ■ ■ ■ page.tsx # Live monitoring dashboard
■ ■ ■ ■ session/
■ ■ ■ ■ page.tsx # Start session screen
■ ■ ■ ■ history/
■ ■ ■ ■ page.tsx # Session history
■ ■ ■ ■ course/
■ ■ ■ ■ page.tsx # Course analytics (trend view)
■ ■ ■ ■ components/
■ ■ ■ ■ EngagementGauge.tsx
■ ■ ■ ■ EmotionChart.tsx
■ ■ ■ ■ SessionTimer.tsx
■ ■ ■ ■ lib/
■ ■ ■ ■ api.ts # All API calls centralised here
■ ■ ■ ■ data/
■ ■ ■ ■ download_datasets.sh # Script to download FER-2013/DAiSEE
■ ■ ■ ■ docker-compose.yml
■ ■ ■ ■ requirements.txt
```

■■■ README.md

Setup Commands — Run These Once

```
# Terminal — run from project root
## STEP 1 — Create project and GitHub repo
mkdir classsense && cd classsense
git init
git remote add origin https://github.com/YOUR_USERNAME/classsense-fyp.git

## STEP 2 — Python virtual environment
python3.10 -m venv venv
source venv/bin/activate # Mac/Linux
# venv\Scripts\activate # Windows

## STEP 3 — Install all Python packages
pip install opencv-python mediapipe torch torchvision
pip install deep-sort-realtime fer
pip install fastapi uvicorn[standard] sqlalchemy psycpg2-binary
pip install python-jose[cryptography] passlib[bcrypt]
pip install reportlab pandas numpy Pillow python-multipart
pip freeze > requirements.txt

## STEP 4 — Frontend setup
npx create-next-app@latest frontend --typescript --tailwind --eslint
cd frontend
npm install recharts axios
```

Datasets — Download Before You Start

Dataset	What It Contains	How to Get It	Used For
FER-2013	35,887 labeled face images, 7 emotion classes	kaggle datasets download -d msambare/fer2013	Training & fine-tuning emotion model
DAiSEE	Real classroom engagement videos, labeled (engaged/bored/confused/frustrated)	Email daisee@iitroorkee.ac.in — free for academic use, takes 1-2 days	Testing full pipeline on real classroom data
AffectNet	450K+ face images, 8 emotion classes	affectnet.org — free academic request form	Optional: improve emotion model accuracy
University Videos	Your own classroom footage	Meeting with IT/Dean (already arranged)	Demo, validation, fine-tuning on local context

Start today with FER-2013 from Kaggle (immediate download). Email your progress weekly. Do NOT wait for university videos to start building. University videos are for reference only — all development uses public datasets.

Face Detection

Find every

What This Phase Does

Face detection is the very first step. We give it a video frame (a single image from the camera) and it tells us: here are all the faces I found, and here is where each face is located (x position, y position, width, height). We call this location a 'bounding box'.

Why MediaPipe? It runs in real time on a normal laptop CPU — no GPU needed. It's 5 meters away, which covers most classroom setups. It is maintained by Google. MTCNN is more accurate but 40x slower — not usable for real time.

File to Create: ml/detection/detector.py

```
# ml/detection/detector.py
import cv2
import mediapipe as mp
import numpy as np
from dataclasses import dataclass
from typing import List, Tuple

@dataclass
class Detection:
    # Stores one detected face
    x1: int # left edge of face box
    y1: int # top edge of face box
    x2: int # right edge
    y2: int # bottom edge
    confidence: float # how sure the model is (0.0 to 1.0)

    def to_xywh(self):
        # Returns [x, y, width, height] format (needed by SORT tracker)
        return [self.x1, self.y1, self.x2 - self.x1, self.y2 - self.y1]

    def to_ltrb(self):
        # Returns [left, top, right, bottom] format
        return [self.x1, self.y1, self.x2, self.y2]

class FaceDetector:
    def __init__(self, confidence_threshold: float = 0.6):
        # model_selection=1 means detect faces up to 5 metres away
        # Use model_selection=0 for faces within 2 metres (closer shots)
```

```
self.detector = mp.solutions.face_detection.FaceDetection(
    model_selection=1,
    min_detection_confidence=confidence_threshold
)
self.confidence_threshold = confidence_threshold

def detect(self, frame: np.ndarray) -> List[Detection]:
    # frame: BGR image from OpenCV (the raw camera/video frame)
    # Returns: list of Detection objects, one per face found

    h, w = frame.shape[:2]
    # MediaPipe needs RGB, OpenCV gives BGR – convert
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = self.detector.process(rgb_frame)

    detections = []
    if not results.detections:
        return detections # no faces found, return empty list

    for detection in results.detections:
        # MediaPipe gives relative coordinates (0.0 to 1.0)
        # Multiply by frame width/height to get pixel coordinates
        box = detection.location_data.relative_bounding_box
        x1 = max(0, int(box.xmin * w))
        y1 = max(0, int(box.ymin * h))
        x2 = min(w, int((box.xmin + box.width) * w))
        y2 = min(h, int((box.ymin + box.height) * h))
        conf = detection.score[0]

        if conf >= self.confidence_threshold:
            detections.append(Detection(x1, y1, x2, y2, conf))

    return detections

## HOW TO TEST THIS FILE – run: python ml/detection/detector.py
if __name__ == '__main__':
    detector = FaceDetector()
    cap = cv2.VideoCapture(0) # 0 = webcam, or use 'path/to/video.mp4'
    while True:
        ret, frame = cap.read()
        if not ret: break
        faces = detector.detect(frame)
        for f in faces:
            cv2.rectangle(frame, (f.x1, f.y1), (f.x2, f.y2), (0,255,0), 2)
        cv2.imshow('Face Detection Test', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'): break
    cap.release()
    cv2.destroyAllWindows()
```

MILESTONE 1: Run the test at the bottom of detector.py. You should see every face in your webcam or video. If fewer than 70% of visible faces are detected, lower the `confidence_threshold` to 0.5. If too many false positives (random things being detected as faces), increase the `confidence_threshold` to 0.9.

Face Tracking — Anonymous IDs

Give each

What This Phase Does

Without tracking, each video frame is independent — the same student could be detected 300 times as 300 different people. Tracking connects detections across frames so that Student A in frame 1 is still 'ID-014' in frame 500 even if they briefly looked away or were temporarily blocked. This ID is anonymous — just a number, no name, no face image stored.

Why DeepSORT? It handles the classroom scenario well — student blocked by others, and DeepSORT re-identifies them when they reappear. Whenever a face disappears briefly. The library 'deep-sort-realtime' is recommended — use it directly instead of implementing from scratch.

File to Create: ml/tracking/tracker.py

```
# ml/tracking/tracker.py
from deep_sort_realtime.deepsort_tracker import DeepSort
import numpy as np
from dataclasses import dataclass
from typing import List
from ml.detection.detector import Detection

@dataclass
class TrackedFace:
    track_id: int # anonymous ID — e.g. 14 becomes 'ID-014' on screen
    x1: int
    y1: int
    x2: int
    y2: int

    def get_label(self) -> str:
        return f'ID-{{self.track_id:03d}}' # formats as ID-001, ID-014, etc.

    def to_ltrb(self):
        return [self.x1, self.y1, self.x2, self.y2]

    def get_roi(self, frame):
        # Crop just the face area from the full frame
        return frame[self.y1:self.y2, self.x1:self.x2]

class FaceTracker:
```

```
def __init__(self):
    self.tracker = DeepSort(
        max_age=30, # Keep ID alive for 30 frames even if not detected
        n_init=3, # Need 3 consecutive detections to confirm a track
        nms_max_overlap=0.7, # Remove duplicate boxes above 70% overlap
    )

    def update(self, detections: List[Detection], frame) -> List[TrackedFace]:
        # Convert our Detection objects to the format DeepSORT expects:
        # list of ([x, y, w, h], confidence, 'face')
        raw = [
            (det.to_xywh(), det.confidence, 'face')
            for det in detections
        ]
        tracks = self.tracker.update_tracks(raw, frame=frame)

        tracked_faces = []
        for track in tracks:
            if not track.is_confirmed(): # skip unconfirmed tracks
                continue
            ltrb = track.to_ltrb()
            x1, y1, x2, y2 = [int(v) for v in ltrb]
            # Clamp to frame boundaries
            h, w = frame.shape[:2]
            x1, y1 = max(0, x1), max(0, y1)
            x2, y2 = min(w, x2), min(h, y2)
            tracked_faces.append(TrackedFace(track.track_id, x1, y1, x2, y2))

        return tracked_faces

## HOW TO TEST THIS FILE – run: python ml/tracking/tracker.py
if __name__ == '__main__':
    import cv2
    from ml.detection.detector import FaceDetector
    detector = FaceDetector()
    tracker = FaceTracker()
    cap = cv2.VideoCapture('path/to/test_video.mp4')
    while True:
        ret, frame = cap.read()
        if not ret: break
        detections = detector.detect(frame)
        tracked = tracker.update(detections, frame)
        for face in tracked:
            cv2.rectangle(frame, (face.x1, face.y1), (face.x2, face.y2), (255,165,0), 2)
            cv2.putText(frame, face.get_label(), (face.x1, face.y1 - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255,165,0), 1)
        cv2.imshow('Tracking Test', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'): break
    cap.release()
```

MILESTONE 2: Run the tracking test. Each face should have a stable ID that does NOT change even when the student briefly looks away. If IDs keep changing, increase max_age from 30 to 60.

Emotion Recognition

Classify ea

What This Phase Does

We take the cropped face region for each tracked student and pass it through a deep learning model that outputs an emotion label. We start with a quick-start approach using the 'fer' library (which wraps a pre-trained model) so the pipeline works immediately. In Phase 10, we replace this with our own fine-tuned MobileNetV2 for better accuracy.

FER-2013 Raw Emotion	ClassSense Mapping	Reasoning
neutral, happy, surprise	ATTENTIVE (score: 1.0)	Student is calm, focused, or positively responsive
fear, sad	CONFUSED (score: 0.5)	Worry and sadness correlate with confusion in learning context
angry, disgust	DISTRACTED (score: 0.0)	Strong negative emotion signals disengagement from content

File to Create: ml/emotion/classifier.py

```
# ml/emotion/classifier.py
import cv2
import numpy as np
from fer import FER
from typing import Optional

# Mapping from raw FER labels to ClassSense 3-class system
EMOTION_MAP = {
    'neutral': 'attentive',
    'happy': 'attentive',
    'surprise': 'attentive',
    'fear': 'confused',
    'sad': 'confused',
    'angry': 'distracted',
    'disgust': 'distracted',
}

EMOTION_SCORES = {
    'attentive': 1.0,
    'confused': 0.5,
    'distracted': 0.0,
}
```

```
class EmotionClassifier:
    def __init__(self):
        # mtcnn=False because we already did face detection – skip it here
        self.detector = FER(mtcnn=False)

    def classify(self, face_roi: np.ndarray) -> dict:
        # face_roi: a cropped face image (numpy array, BGR colour)
        # Returns a dict with: emotion label, score, and raw probabilities

        if face_roi is None or face_roi.size == 0:
            return self._default_result()

        # FER needs the face to be at least 48x48 pixels
        if face_roi.shape[0] < 48 or face_roi.shape[1] < 48:
            face_roi = cv2.resize(face_roi, (64, 64))

        # Run emotion detection
        emotions = self.detector.detect_emotions(face_roi)

        if not emotions:
            return self._default_result()

        # Get the emotion with highest probability
        raw_emotion = self.detector.top_emotion(face_roi)
        if raw_emotion is None or raw_emotion[0] is None:
            return self._default_result()

        fer_label, fer_confidence = raw_emotion
        classsense_label = EMOTION_MAP.get(fer_label, 'attentive')

        return {
            'emotion': classsense_label, # 'attentive' / 'confused' / 'distracted'
            'score': EMOTION_SCORES[classsense_label], # 1.0 / 0.5 / 0.0
            'fer_label': fer_label, # original FER label for debugging
            'confidence': float(fer_confidence), # model confidence
        }

    def _default_result(self) -> dict:
        # Return neutral if we cannot classify (safe default)
        return {'emotion': 'attentive', 'score': 0.5, 'fer_label': 'neutral', 'confidence': 0.0}

## HOW TO TEST – run: python ml/emotion/classifier.py
if __name__ == '__main__':
    import cv2
    from ml.detection.detector import FaceDetector
    from ml.tracking.tracker import FaceTracker
    classifier = EmotionClassifier()
    detector = FaceDetector()
    tracker = FaceTracker()
    COLOURS = {'attentive': (0,200,0), 'confused': (0,165,255), 'distracted': (0,0,220)}
```

```
cap = cv2.VideoCapture('path/to/test_video.mp4')
while True:
    ret, frame = cap.read()
    if not ret: break
    detections = detector.detect(frame)
    tracked = tracker.update(detections, frame)
    for face in tracked:
        roi = face.get_roi(frame)
        result = classifier.classify(roi)
        colour = COLOURS[result['emotion']]
        cv2.rectangle(frame, (face.x1, face.y1), (face.x2, face.y2), colour, 2)
        cv2.putText(frame, f"{face.get_label()} {result['emotion']}",
                    (face.x1, face.y1-8), cv2.FONT_HERSHEY_SIMPLEX, 0.45, colour, 1)
    cv2.imshow('Emotion Test', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break
cap.release()
```

MILESTONE 3: You should see coloured labels on every tracked face. If a face is confused, red = distracted. Accuracy at this stage is ~55-65% — that's a good start. 10. The goal now is a working pipeline, not perfect accuracy.

Engagement Scoring

Combine a

What This Phase Does

Each student has an emotion label and (later) a gaze direction. We combine these into a single number per student (0.0 = fully disengaged, 1.0 = fully engaged). Then we average across all students to get the class percentage shown on the dashboard.

```
# Engagement formula
# The formula (simple and explainable to supervisors):
# engagement_per_student = (0.6 x attention_score) + (0.4 x emotion_score)
# class_engagement_pct = average(all student scores) x 100

# In Phase 5 we add gaze. Until then:
# engagement_per_student = emotion_score only
```

File to Create: ml/engagement/scorer.py

```
# ml/engagement/scorer.py
from dataclasses import dataclass, field
from typing import List, Dict
from collections import deque

@dataclass
class StudentEngagement:
    track_id: int
    emotion: str
    emotion_score: float
    gaze_score: float = 1.0 # default: looking forward (updated in Phase 5)

    @property
    def engagement_score(self) -> float:
        # Weighted combination: 60% gaze + 40% emotion
        return (0.6 * self.gaze_score) + (0.4 * self.emotion_score)

@dataclass
class ClassEngagement:
    # Output object – this is what gets sent to the dashboard
    engagement_pct: float # e.g. 87.3
    student_count: int # number of students detected
    attentive_count: int
    confused_count: int
    distracted_count: int
```

```
student_details: List[dict] = field(default_factory=list)

def to_dict(self) -> dict:
    return {
        'engagement_pct': round(self.engagement_pct, 1),
        'student_count': self.student_count,
        'distribution': {
            'attentive': self.attentive_count,
            'confused': self.confused_count,
            'distracted': self.distracted_count,
        }
    }

class EngagementScorer:
    def __init__(self, smoothing_window: int = 10):
        # smoothing_window: average over last N frames to smooth out noise
        self.history = deque(maxlen=smoothing_window)

    def score_frame(self, students: List[StudentEngagement]) -> ClassEngagement:
        if not students:
            return ClassEngagement(0.0, 0, 0, 0, 0)

        scores = [s.engagement_score for s in students]
        raw_pct = (sum(scores) / len(scores)) * 100

        # Store in history and return smoothed value
        self.history.append(raw_pct)
        smoothed_pct = sum(self.history) / len(self.history)

        return ClassEngagement(
            engagement_pct=smoothed_pct,
            student_count=len(students),
            attentive_count=sum(1 for s in students if s.emotion == 'attentive'),
            confused_count=sum(1 for s in students if s.emotion == 'confused'),
            distracted_count=sum(1 for s in students if s.emotion == 'distracted'),
            student_details=[
                {'id': s.track_id, 'emotion': s.emotion, 'score': s.engagement_score}
                for s in students
            ]
        )
```

MILESTONE 4: The scorer now converts a list of student emotions in a visual test needed — this is pure logic. Write a quick unit test: pass in a list of emotions and you should get engagement_pct around 75.

Gaze & Attention Estimation

Determine

What This Phase Does

We use MediaPipe Face Mesh which gives us 468 3D points on each face. From the nose tip and eye landmarks we calculate the head rotation angle (called yaw). If yaw is small (head facing forward) → student is attentive. If yaw is large (head turned sideways) → student is looking away.

File to Create: ml/gaze/estimator.py

```
# ml/gaze/estimator.py
import cv2
import numpy as np
import mediapipe as mp
from typing import Optional

# The 3D reference points of a generic human face (used for head pose math)
FACE_3D_MODEL = np.array([
    [0.0, 0.0, 0.0 ], # Nose tip
    [0.0, -330.0, -65.0 ], # Chin
    [-225.0, 170.0, -135.0 ], # Left eye corner
    [225.0, 170.0, -135.0 ], # Right eye corner
    [-150.0, -150.0, -125.0 ], # Left mouth corner
    [150.0, -150.0, -125.0 ], # Right mouth corner
], dtype=np.float64)

# Face Mesh landmark indices matching the 3D points above
LANDMARK_IDS = [1, 152, 226, 446, 57, 287]

class GazeEstimator:
    def __init__(self):
        self.face_mesh = mp.solutions.face_mesh.FaceMesh(
            max_num_faces=40, # handle large classrooms
            refine_landmarks=True, # more accurate eye landmarks
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5
        )

    def estimate(self, frame: np.ndarray) -> dict:
        # Returns: {track_id_approx: gaze_score} but since we don't have
        # track IDs here, we return a list of gaze scores to be matched
        # by the pipeline. We'll improve this integration in ml/pipeline.py.

        h, w = frame.shape[:2]
```

```
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
results = self.face_mesh.process(rgb)
gaze_scores = []

if not results.multi_face_landmarks:
    return gaze_scores

for face_landmarks in results.multi_face_landmarks:
    # Extract 2D screen positions of our 6 key landmarks
    points_2d = []
    for idx in LANDMARK_IDS:
        lm = face_landmarks.landmark[idx]
        points_2d.append([lm.x * w, lm.y * h])
    points_2d = np.array(points_2d, dtype=np.float64)

    # Camera matrix (estimated from frame size)
    focal = w
    cam_matrix = np.array([
        [focal, 0, w / 2],
        [0, focal, h / 2],
        [0, 0, 1]
    ], dtype=np.float64)

    dist_coeffs = np.zeros((4,1))

    # Solve PnP: find head rotation from 2D/3D point correspondence
    success, rot_vec, _ = cv2.solvePnP(
        FACE_3D_MODEL, points_2d, cam_matrix, dist_coeffs
    )
    if not success:
        gaze_scores.append(0.5) # neutral default
        continue

    # Convert rotation vector to angles
    rot_mat, _ = cv2.Rodrigues(rot_vec)
    angles, _, _, _, _ = cv2.RQDecomp3x3(rot_mat)
    yaw = abs(angles[1]) # left-right head rotation
    pitch = abs(angles[0]) # up-down head rotation

    # Score: 1.0 = looking straight ahead, 0.0 = fully turned away
    if yaw < 15 and pitch < 20:
        gaze_scores.append(1.0) # clearly attentive
    elif yaw < 30:
        gaze_scores.append(0.6) # slightly turned
    elif yaw < 45:
        gaze_scores.append(0.3) # significantly turned
    else:
        gaze_scores.append(0.0) # looking away completely

return gaze_scores
```

MILESTONE 5: Add a test where you tilt your head left and right in f
should drop. Looking straight forward should give score close to 1.0.

Unified ML Pipeline

Combine a

File to Create: ml/pipeline.py

This is the orchestrator — it imports all the previous modules and runs them in order for every frame.

```
# ml/pipeline.py – The orchestrator
import cv2
import numpy as np
from typing import Generator, Dict
from ml.detection.detector import FaceDetector
from ml.tracking.tracker import FaceTracker
from ml.emotion.classifier import EmotionClassifier
from ml.gaze.estimator import GazeEstimator
from ml.engagement.scorer import EngagementScorer, StudentEngagement

class ClassSensePipeline:
    def __init__(self):
        print('Loading ClassSense pipeline...')
        self.detector = FaceDetector(confidence_threshold=0.6)
        self.tracker = FaceTracker()
        self.classifier = EmotionClassifier()
        self.gaze = GazeEstimator()
        self.scorer = EngagementScorer(smoothing_window=10)
        self.frame_count = 0
        print('Pipeline ready.')

    def process_frame(self, frame: np.ndarray) -> dict:
        # INPUT: one video frame (numpy array, BGR colour from OpenCV)
        # OUTPUT: dict with engagement data, ready to send to dashboard

        self.frame_count += 1

        # Skip processing every 5th frame for performance
        # Tracking still runs every frame for smooth boxes
        do_emotion = (self.frame_count % 5 == 0)

        # STEP 1: Detect faces
        detections = self.detector.detect(frame)

        # STEP 2: Track faces (assign/maintain anonymous IDs)
        tracked_faces = self.tracker.update(detections, frame)

        # STEP 3 & 4: Get gaze scores for the full frame
        gaze_scores = self.gaze.estimate(frame)
```

```

# STEP 5: Classify emotions and build student list
students = []
for i, face in enumerate(tracked_faces):
    if do_emotion:
        roi = face.get_roi(frame)
        emotion_result = self.classifier.classify(roi)
    else:
        # Reuse last known emotion (stored in tracker state)
        emotion_result = {'emotion': 'attentive', 'score': 0.5}

    gaze_score = gaze_scores[i] if i < len(gaze_scores) else 0.7

    students.append(StudentEngagement(
        track_id=face.track_id,
        emotion=emotion_result['emotion'],
        emotion_score=emotion_result['score'],
        gaze_score=gaze_score,
    ))

# STEP 6: Compute class engagement
class_eng = self.scorer.score_frame(students)

return {
    'frame_id': self.frame_count,
    'engagement': class_eng.to_dict(),
    'tracked_count': len(tracked_faces),
    'faces': [{'id': f.track_id, 'bbox': f.to_ltrb()} for f in tracked_faces]
}

def process_video(self, video_path: str) -> Generator[dict, None, None]:
    # Process entire video file, yield one result per processed frame
    cap = cv2.VideoCapture(video_path)
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        yield self.process_frame(frame)
    cap.release()

## HOW TO TEST THE FULL PIPELINE
if __name__ == '__main__':
    pipeline = ClassSensePipeline()
    for result in pipeline.process_video('path/to/classroom_video.mp4'):
        if result['frame_id'] % 30 == 0:
            eng = result['engagement']
            print(f"Frame {result['frame_id']}: {eng['engagement_pct']}% engaged | "
                  f"{result['tracked_count']} students | {eng['distribution']}")

```

MILESTONE 6: Run the pipeline test on a DAiSEE video. Every 30 f
'Frame 30: 74.3% engaged | 12 students | {attentive: 9, confused: 2,
ML engine is functional.

Database — PostgreSQL Setup

[Create tab](#)

Install and Create Database

```
# Terminal setup
## Install PostgreSQL (Ubuntu/WSL)
sudo apt-get install postgresql postgresql-contrib
sudo service postgresql start

## Create database and user
sudo -u postgres psql
CREATE DATABASE classsense_db;
CREATE USER classsense_user WITH PASSWORD 'your_password_here';
GRANT ALL PRIVILEGES ON DATABASE classsense_db TO classsense_user;
\q

## Add to backend/.env file (create this file)
DATABASE_URL=postgresql://classsense_user:your_password_here@localhost/classsense_db
SECRET_KEY=your-secret-key-minimum-32-chars
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=480
```

File to Create: backend/database.py

```
# backend/database.py
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
import os

DATABASE_URL = os.getenv('DATABASE_URL',
    'postgresql://classsense_user:password@localhost/classsense_db')

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    # FastAPI dependency – provides a DB session per request
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

File to Create: backend/models/session.py

```
# backend/models/session.py

from sqlalchemy import Column, Integer, Float, String, DateTime, JSON
from sqlalchemy.sql import func
from backend.database import Base

class Session(Base):
    __tablename__ = 'sessions'
    id = Column(Integer, primary_key=True, index=True)
    instructor_id = Column(Integer, nullable=False)
    course_name = Column(String(100))
    time_slot = Column(String(50))
    started_at = Column(DateTime(timezone=True), server_default=func.now())
    ended_at = Column(DateTime(timezone=True), nullable=True)
    status = Column(String(20), default='active') # active | ended

class FrameAnalytic(Base):
    __tablename__ = 'frame_analytics'
    id = Column(Integer, primary_key=True, index=True)
    session_id = Column(Integer, nullable=False)
    timestamp = Column(DateTime(timezone=True), server_default=func.now())
    engagement_pct = Column(Float)
    student_count = Column(Integer)
    distribution = Column(JSON) # {attentive: N, confused: N, distracted: N}

class SessionSummary(Base):
    __tablename__ = 'session_summaries'
    id = Column(Integer, primary_key=True)
    session_id = Column(Integer, unique=True, nullable=False)
    avg_engagement = Column(Float)
    peak_engagement = Column(Float)
    min_engagement = Column(Float)
    avg_students = Column(Float)
    duration_mins = Column(Float)
    report_path = Column(String(200), nullable=True) # path to PDF file

# Aggregated emotion totals for the full session
total_attentive = Column(Integer, default=0)
total_confused = Column(Integer, default=0)
total_distracted = Column(Integer, default=0)
```

Backend — FastAPI Server

API endpo

File to Create: backend/main.py

```
# backend/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect, Depends
from fastapi.middleware.cors import CORSMiddleware
from backend.database import engine, Base
from backend.routers import sessions, analytics
import uvicorn

# Create all database tables on startup
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title='ClassSense API',
    description='Real-time classroom engagement monitoring backend',
    version='1.0.0'
)

# Allow Next.js frontend (localhost:3000) to call this API
app.add_middleware(
    CORSMiddleware,
    allow_origins=['http://localhost:3000'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

app.include_router(sessions.router, prefix='/api/sessions', tags=['Sessions'])
app.include_router(analytics.router, prefix='/api/analytics', tags=['Analytics'])

@app.get('/health')
def health_check():
    return {'status': 'ok', 'service': 'ClassSense API'}

if __name__ == '__main__':
    uvicorn.run('backend.main:app', host='0.0.0.0', port=8000, reload=True)
```

File to Create: backend/routers/sessions.py

```
# backend/routers/sessions.py
from fastapi import APIRouter, Depends, UploadFile, File, WebSocket, WebSocketDisconnect
from sqlalchemy.orm import Session
```

[illegible]

[illegible]

File to Create: backend/services/ml_runner.py

```
# backend/services/ml_runner.py
import cv2, numpy as np
from ml.pipeline import ClassSensePipeline

class MLRunner:
    def __init__(self):
        self.pipeline = ClassSensePipeline()

    def process_video(self, video_path: str):
        # Generator — yields one result per processed frame
        return self.pipeline.process_video(video_path)
```

```
def process_bytes(self, frame_bytes: bytes) -> dict:
    # Decode raw bytes to numpy array then process
    arr = np.frombuffer(frame_bytes, dtype=np.uint8)
    frame = cv2.imdecode(arr, cv2.IMREAD_COLOR)
    if frame is None:
    return {'engagement': {'engagement_pct': 0, 'student_count': 0, 'distribution': {}}}
    return self.pipeline.process_frame(frame)
```

MILESTONE 7: Start the server with 'uvicorn backend.main:app --re your browser. You should see the Swagger UI with all your endpoint should return {'status': 'ok'}.

Frontend — Next.js Dashboard

Build all 6

Screen Build Order (Important)

Build screens in this exact order. Each one depends on the previous.

Order	Screen	File	What to Build
1	Login	app/page.tsx	Email + password form. On submit, call POST /auth/login, save JWT token, redirect to dashboard
2	Start Session	app/session/page.tsx	Dropdown for course name + time slot. On submit, call POST /sessions/start, get session_id
3	Live Dashboard	app/dashboard/page.tsx	The main screen. Big engagement % number, emotion distribution bars, student count, End Session button
4	Session Summary	app/summary/[id]/page.tsx	Pie chart of attention distribution for one session. Download PDF/CSV buttons
5	Session History	app/history/page.tsx	Table of all past sessions with engagement %, date, View Details + Export buttons
6	Course Analytics	app/course/[id]/page.tsx	Line chart showing engagement trend across multiple sessions of same course

File: app/dashboard/page.tsx — The Core Screen

```
# frontend/app/dashboard/page.tsx
'use client'
import { useEffect, useState, useRef } from 'react'
import { useSearchParams } from 'next/navigation'
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts'

interface EngagementData {
  engagement_pct: number
  student_count: number
  distribution: { attentive: number; confused: number; distracted: number }
}

export default function Dashboard() {
  const sessionId = useSearchParams().get('session_id')
  const [engagement, setEngagement] = useState<EngagementData | null>(null)
  const [history, setHistory] = useState<{pct: number; time: string}[]>([])
```

```

const [timer, setTimer] = useState(0)
const wsRef = useRef<WebSocket | null>(null)

useEffect(() => {
  if (!sessionId) return

  // Connect to backend WebSocket
  const ws = new WebSocket(`ws://localhost:8000/api/sessions/${sessionId}/stream`)
  wsRef.current = ws

  ws.onmessage = (event) => {
    const data: EngagementData = JSON.parse(event.data)
    setEngagement(data)
    setHistory(prev => [...prev.slice(-30), {
      pct: data.engagement_pct,
      time: new Date().toLocaleTimeString()
    }])
  }

  // Timer
  const interval = setInterval(() => setTimer(t => t + 1), 1000)
  return () => { ws.close(); clearInterval(interval) }
}, [sessionId])

const formatTime = (s: number) =>
`${Math.floor(s/60).toString().padStart(2, '0')}:${(s%60).toString().padStart(2, '0')}`

const endSession = async () => {
  wsRef.current?.close()
  await fetch(`http://localhost:8000/api/sessions/${sessionId}/end`, { method: 'POST' })
  window.location.href = `/summary/${sessionId}`
}

return (
  <div className='min-h-screen bg-gray-950 text-white p-6'>
    { /* Header */ }
    <div className='flex justify-between items-center mb-8'>
      <h1 className='text-2xl font-bold text-blue-400'>ClassSense – Live Session</h1>
      <span className='text-gray-400 font-mono'>{formatTime(timer)}</span>
    </div>

    { /* Main engagement number */ }
    <div className='text-center my-10'>
      <div className='text-8xl font-black text-white'>
        {engagement ? `${engagement.engagement_pct.toFixed(1)}%` : '-'}
      </div>
      <p className='text-gray-400 text-xl mt-2'>Currently Engaged</p>
      <p className='text-gray-500 mt-1'>{engagement?.student_count || 0} students detected</p>
    </div>

    { /* Emotion distribution bars */ }
    {engagement && (

```

```

<div className='grid grid-cols-3 gap-4 my-8'>
  [[
    { label: 'Attentive', count: engagement.distribution.attentive, colour: 'bg-green-500' },
    { label: 'Confused', count: engagement.distribution.confused, colour: 'bg-yellow-500' },
    { label: 'Distracted', count: engagement.distribution.distracted, colour: 'bg-red-500' },
  ].map(({ label, count, colour }) => (
    <div key={label} className='bg-gray-800 rounded-xl p-5'>
      <p className='text-gray-400 text-sm'>{label}</p>
      <p className='text-3xl font-bold mt-1'>{count}</p>
      <div className='mt-3 h-2 bg-gray-700 rounded-full'>
        <div className={`h-2 ${colour} rounded-full`}
          style={{ width: `${engagement.student_count > 0 ? (count/engagement.student_count)*100 : 0}%` }}/>
      </div>
    </div>
  )))
</div>

{/* Trend line chart */}
<div className='bg-gray-800 rounded-xl p-6 my-6'>
  <h2 className='text-gray-400 mb-4'>Engagement Trend</h2>
  <ResponsiveContainer width='100%' height={200}>
    <LineChart data={history}>
      <XAxis dataKey='time' tick={{ fill: '#6B7280', fontSize: 10 }} />
      <YAxis domain={[0, 100]} tick={{ fill: '#6B7280', fontSize: 10 }} />
      <Tooltip />
      <Line type='monotone' dataKey='pct' stroke='#3B82F6' strokeWidth={2} dot={false} />
    </LineChart>
  </ResponsiveContainer>
</div>

{/* End session button */}
<div className='text-center mt-8'>
  <button onClick={endSession}
    className='bg-red-600 hover:bg-red-700 px-12 py-3 rounded-lg font-bold text-lg'>
    End Session
  </button>
</div>
</div>
)
}

```

MILESTONE 8: Run 'npm run dev' and open <http://localhost:3000/dashboard> backend running, you should see the live engagement number updating in the end-to-end demo.

Model Fine-tuning

Replace the

Why Fine-tune?

The 'fer' library gives about 55-65% accuracy. Fine-tuning MobileNetV2 on FER-2013 (mapped to our 3 classes) gives 68-75% accuracy with similar inference speed. Do this AFTER the full pipeline is working — accuracy improvements should never block pipeline development.

[illegible]

```
total_loss, correct, total = 0, 0, 0
for images, labels in train_loader:
    images, labels = images.to(device), labels.to(device)
    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()
    total_loss += loss.item()
    correct += (outputs.argmax(1) == labels).sum().item()
    total += labels.size(0)
print(f'Epoch {epoch+1}: Loss={total_loss/len(train_loader):.3f} Acc={100*correct/total:.1f}%')

# Save the trained weights
torch.save(model.state_dict(), 'ml/emotion/model_weights/mobilenet_fer.pth')
print('Model saved.')
```

Update classifier.py to Use Trained Model

```
# ml/emotion/classifier.py – add fine-tuned classifier
# Add this class to ml/emotion/classifier.py alongside EmotionClassifier

import torch, torchvision
import torch.nn as nn
from torchvision import transforms

CLASSES = ['attentive', 'confused', 'distracted']
SCORES = {'attentive': 1.0, 'confused': 0.5, 'distracted': 0.0}

class FineTunedClassifier:
    def __init__(self, weights_path='ml/emotion/model_weights/mobilenet_fer.pth'):
        self.model = torchvision.models.mobilenet_v2(pretrained=False)
        self.model.classifier[1] = nn.Linear(self.model.last_channel, 3)
        self.model.load_state_dict(torch.load(weights_path, map_location='cpu'))
        self.model.eval()
        self.transform = transforms.Compose([
            transforms.ToPILImage(),
            transforms.Grayscale(num_output_channels=3),
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
        ])

    def classify(self, face_roi) -> dict:
        if face_roi is None or face_roi.size == 0:
            return {'emotion': 'attentive', 'score': 0.5, 'confidence': 0.0}
        with torch.no_grad():
            tensor = self.transform(face_roi).unsqueeze(0)
            probs = torch.softmax(self.model(tensor), dim=1)[0]
            idx = probs.argmax().item()
            label = CLASSES[idx]
```

```
return {'emotion': label, 'score': SCORES[label], 'confidence': probs[idx].item()}

# In ml/pipeline.py – swap EmotionClassifier() for FineTunedClassifier() after training
```

Reports — PDF & CSV Export

[Generate](#)

File to Create: backend/services/report.py

```
# backend/services/report.py
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.lib import colors
import pandas as pd, os

def generate_pdf_report(session_id: int, summary: dict, frames: list) -> str:
    output_path = f'reports/session_{session_id}_report.pdf'
    os.makedirs('reports', exist_ok=True)

    doc = SimpleDocTemplate(output_path, pagesize=A4)
    styles = getSampleStyleSheet()
    story = []

    story.append(Paragraph('ClassSense Session Report', styles['Title']))
    story.append(Paragraph(f'Session ID: {session_id}', styles['Normal']))
    story.append(Spacer(1, 12))

    # Summary table
    summary_data = [
        ['Average Engagement', f'{summary['avg_engagement']:.1f}%'],
        ['Peak Engagement', f'{summary['peak_engagement']:.1f}%'],
        ['Min Engagement', f'{summary['min_engagement']:.1f}%'],
        ['Average Students', str(int(summary['avg_students']))],
        ['Duration', f'{summary['duration_mins']:.0f} minutes'],
    ]
    tbl = Table(summary_data, colWidths=[200, 200])
    tbl.setStyle(TableStyle([
        ('BACKGROUND', (0,0), (0,-1), colors.HexColor('#E3F2FD')),
        ('GRID', (0,0), (-1,-1), 0.5, colors.grey),
        ('FONTNAME', (0,0), (0,-1), 'Helvetica-Bold'),
        ('PADDING', (0,0), (-1,-1), 8),
    ]))
    story.append(tbl)
    doc.build(story)
    return output_path

def generate_csv_report(session_id: int, frames: list) -> str:
    output_path = f'reports/session_{session_id}_data.csv'
    os.makedirs('reports', exist_ok=True)
    df = pd.DataFrame([
        'timestamp': f['timestamp'],
```

```
'engagement': f['engagement_pct'],
'students': f['student_count'],
'attentive': f['distribution'].get('attentive', 0),
'confused': f['distribution'].get('confused', 0),
'distracted': f['distribution'].get('distracted', 0),
} for f in frames])
df.to_csv(output_path, index=False)
return output_path
```

Report Download Endpoint — add to analytics.py

```
# backend/routers/analytics.py
from fastapi import APIRouter
from fastapi.responses import FileResponse
from backend.services.report import generate_pdf_report, generate_csv_report

router = APIRouter()

@router.get('/{session_id}/report/pdf')
def download_pdf(session_id: int, db=Depends(get_db)):
    summary = db.query(SessionSummary).filter_by(session_id=session_id).first()
    frames = db.query(FrameAnalytic).filter_by(session_id=session_id).all()
    path = generate_pdf_report(session_id, summary.__dict__, [f.__dict__ for f in frames])
    return FileResponse(path, media_type='application/pdf', filename=f'session_{session_id}.pdf')

@router.get('/{session_id}/report/csv')
def download_csv(session_id: int, db=Depends(get_db)):
    frames = db.query(FrameAnalytic).filter_by(session_id=session_id).all()
    path = generate_csv_report(session_id, [f.__dict__ for f in frames])
    return FileResponse(path, media_type='text/csv', filename=f'session_{session_id}.csv')
```

Authentication — JWT Login

Secure ins

File to Create: backend/auth.py

```
# backend/auth.py
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from jose import JWTError, jwt
from passlib.context import CryptContext
from datetime import datetime, timedelta
import os

SECRET_KEY = os.getenv('SECRET_KEY', 'change-this-in-production')
ALGORITHM = 'HS256'
EXPIRE_MIN = 480 # 8 hours – one full teaching day

pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto')
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='/auth/token')
router = APIRouter()

# Hardcoded users for MVP (replace with DB in production)
USERS = {
    'instructor@classsense.com': {
        'hashed_password': pwd_context.hash('instructor123'),
        'role': 'instructor'
    }
}

def create_token(data: dict) -> str:
    payload = data.copy()
    payload['exp'] = datetime.utcnow() + timedelta(minutes=EXPIRE_MIN)
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload.get('sub')
    except JWTError:
        raise HTTPException(status_code=401, detail='Invalid or expired token')

@router.post('/token')
def login(form: OAuth2PasswordRequestForm = Depends()):
    user = USERS.get(form.username)
    if not user or not pwd_context.verify(form.password, user['hashed_password']):
        raise HTTPException(status_code=401, detail='Incorrect email or password')
    token = create_token({'sub': form.username, 'role': user['role']})
```

```
return {'access_token': token, 'token_type': 'bearer'}
```

Docker & Deployment

Package e

File to Create: docker-compose.yml

```
# docker-compose.yml
version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: classsense_db
      POSTGRES_USER: classsense_user
      POSTGRES_PASSWORD: classsense_pass
    ports:
      - '5432:5432'
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build: .
    command: uvicorn backend.main:app --host 0.0.0.0 --port 8000
    ports:
      - '8000:8000'
    environment:
      DATABASE_URL: postgresql://classsense_user:classsense_pass@db/classsense_db
    depends_on:
      - db
    volumes:
      - ./reports:/app/reports

  frontend:
    build: ./frontend
    ports:
      - '3000:3000'
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000
    depends_on:
      - backend

    volumes:
      postgres_data:
```

```
# Terminal — Docker commands
## Build and start everything
docker-compose up --build
```

```
## Run in background
docker-compose up -d

## Stop everything
docker-compose down

## View logs
docker-compose logs -f backend
```

Testing & FYP Evaluation

What to m

Quantitative Tests to Run

Test	How to Run	What to Report	Target
Emotion accuracy	Run classifier on FER-2013 test set (3,589 images). Compare predicted vs true labels.	Accuracy % per class + overall	>65%
Engagement accuracy	Run pipeline on 5 DAiSEE videos with known labels. Compare engagement score to ground truth.	Mean Absolute Error (MAE)	<15% MAE
Inference speed	Process 100 frames, measure total time with time.time()	Milliseconds per frame	<150ms/frame on CPU
Student scalability	Test with 5, 15, 30 faces simultaneously (use crowd images)	FPS at each count	>5 FPS at 30 faces
Privacy check	Inspect DB after a session — no names or raw images should exist	Pass/Fail audit	100% pass

Usability Test (Instructor UX)

Ask 2 instructors (or your supervisor) to complete these tasks without your help:

- Login to the system
- Start a new session for CS101
- Identify which metric means 'confused students'
- End the session and download the PDF report
- Find a past session from 3 days ago

Record: time per task, errors made, questions asked. Anything that confuses them → fix it.

FYP Submission Checklist

- GitHub repository — clean, documented README, daily commits visible in history

- Live demo video (3–4 minutes) showing: login → start session → live dashboard updating → end session → download report
- Performance evaluation report — accuracy, latency, scalability table
- User testing results — 2+ instructors, task completion rates
- All Activities (1–5) documents — already done
- Final presentation slides (aligned with Business Model Canvas from Activity 5)
- Docker setup working — supervisor can run 'docker-compose up' and it works

Appendix A — Complete Dataset Guide

Dataset	Size	Get It	Use In Phase
FER-2013	35,887 images	kaggle datasets download -d msambare/fer2013	Phase 3 (quick start) + Phase 10 (fine-tuning)
DAiSEE	9,068 video clips, labeled engagement	Email daisee@iitroorkee.ac.in	Phase 6 testing + Phase 14 evaluation
AffectNet	450K+ images	affectnet.org form	Phase 10 optional improvement
University Videos	10-15 sessions	IT coordinator meeting	Phase 9 demo + Phase 14 final evaluation

Appendix B — How to Use This Document with AI Assistants

When asking ChatGPT, Claude, Cursor, or any AI assistant for help with this project, start your message with the following context block. This gives the AI everything it needs to give accurate, project-specific answers:

```
# Copy-paste this block when asking any AI for help
## Context for AI Assistant
I am building ClassSense – a real-time classroom engagement monitoring system for my Final Year Project
at Iqra University (Computer Science, Batch 2022).

## Tech Stack
- ML: Python 3.10, OpenCV, MediaPipe, PyTorch (MobileNetV2), SORT/DeepSORT
- Backend: FastAPI, SQLAlchemy, PostgreSQL
- Frontend: Next.js 14, TypeScript, TailwindCSS, Recharts
- Reports: ReportLab (PDF), Pandas (CSV)
- Auth: JWT (python-jose), bcrypt (passlib)
- Deployment: Docker Compose

## System Pipeline
Camera/Video → Face Detection (MediaPipe) → Tracking (DeepSORT, anonymous IDs) →
Emotion Recognition (MobileNetV2 fine-tuned on FER-2013) → Gaze (MediaPipe Face Mesh) →
Engagement Score → FastAPI backend → WebSocket → Next.js dashboard

## Emotion Classes
3 classes: attentive (score 1.0), confused (score 0.5), distracted (score 0.0)
Mapped from FER-2013: neutral/happy/surprise=attentive, fear/sad=confused,
angry/disgust=distracted

## Privacy Rules
No face images stored, no names, no biometrics. Only anonymous numeric IDs and engagement %.

## Current Phase
```

[REPLACE WITH YOUR CURRENT PHASE NUMBER AND WHAT YOU ARE WORKING ON]

My Question

[YOUR SPECIFIC QUESTION HERE]

ClassSense — Master Implementation Document | Version 1.0 | FYP 2025-2026 | Iqra University, Department of Computer Science | Sameer Shah (62662) & Ismail Haroon (63188) | Supervisor: Ms. Zuha Soomro